Установка SQLite

1. Скачивание SQLite:

   - Перейдите на официальный сайт (https://www.sqlite.org/download.html).

   - Скачайте пакет, соответствующий вашей операционной системе. Для Windows это sqlite-tools-win-x64-3460100.zip.

2. Установка:

   - Распакуйте скачанный архив в удобное для вас место, например, на C:\sqlite.

3. Добавление в PATH (опционально):

   - Чтобы запускать SQLite из командной строки, добавьте путь к C:\sqlite в переменную окружения PATH.

   - Для этого:

     - Нажмите «Параметры» -> «Система» -> «О программе» -> «Дополнительные параметры системы».

     - На вкладке «Дополнительно» нажмите «Переменные среды».

     - Найдите переменную Path, выберите ее и нажмите «Изменить».

     - Добавьте путь к папке C:\sqlite.

Установка SQLite

4. Проверка установки:

- Откройте командную строку и введите sqlite3. Если вы видите сообщение о версии SQLite, установка прошла успешно.

Создание базы данных

1. Создание базы данных:

- В командной строке выполните команду:

sqlite3 название_базы.db

- Это создаст файл название_базы.db в текущем каталоге и откроет оболочку SQLite.

2. Создание таблиц:

Выполните команды для создания таблиц.

3. Добавление данных:

Выполните команды для вставки данных в таблицы.

4. Сохранение и выход:

- Для сохранения базы данных и выхода из SQLite напишите .exit.

Разработка приложения на C#

1. Создание проекта:

- Откройте Visual Studio и создайте новый проект типа "Windows Forms App (.NET Framework)".

- Назовите проект .

2. Добавление NuGet-пакета SQLite:

- Откройте "Управление NuGet-пакетами" (правый клик на проект -> "Управление NuGet-пакетами").

- Найдите и установите пакет System.Data.SQLite.

Разработка приложения на C#

3. Подключение к базе данных:

В главном файле формы (Form1.cs) добавьте следующий код для подключения к базе данных:

```
using System.Data.SQLite;
  public partial class Form1 : Form
  {
     private SQLiteConnection sqliteConn;
     public Form1()
     {
        InitializeComponent();
        sqliteConn = new SQLiteConnection("Data Source=university.db;Version=3;");
        sqliteConn.Open();
     }
```

Разработка приложения на C#

```
    private void Form1_FormClosed(object sender, FormClosedEventArgs e)
    {
        sqliteConn.Close();
    }
}
```

4. Создание интерфейса:

- Добавьте элементы управления (например, DataGridView, Buttons, `TextBoxes`) для отображения данных.

- Используйте кнопки для загрузки данных из базы данных.

Разработка приложения на C#

5. Выполнение SQL-запросов:

Пример кода для загрузки студентов в DataGridView:

```
private void LoadStudents()
  {
     string query = "SELECT * FROM Students";
     SQLiteDataAdapter adapter = new SQLiteDataAdapter(query, sqliteConn);
     DataTable studentsTable = new DataTable();
     adapter.Fill(studentsTable);
     dataGridView1.DataSource = studentsTable;
  }
```

Разработка локальной базы данных на основе SQLite

Создадим базу Spravka и таблицы БД

EIzmeren

ID_E	INTEGER
NameE	TEXT

Proizvod

ID_P	INTEGER
NameP	TEXT
AdresP	TEXT

Tovar

ID_T	INTEGER
NameT	TEXT
FK_E	INTEGER
FK_P	INTEGER

Magazin

ID_M	INTEGER
NameM	TEXT
AdresM	TEXT

Spravka

ID_S	INTEGER
Kolvo	REAL
Cena	REAL
FK_T	INTEGER
FK_M	INTEGER

Разработка клиентской части для локальной базы

Разработаем клиентское приложение для нашей созданной базы для этого создадим папку для проекта и перенесем в эту папку созданную базу данных. Клиентское приложение будем разрабатывать по типу MDI.

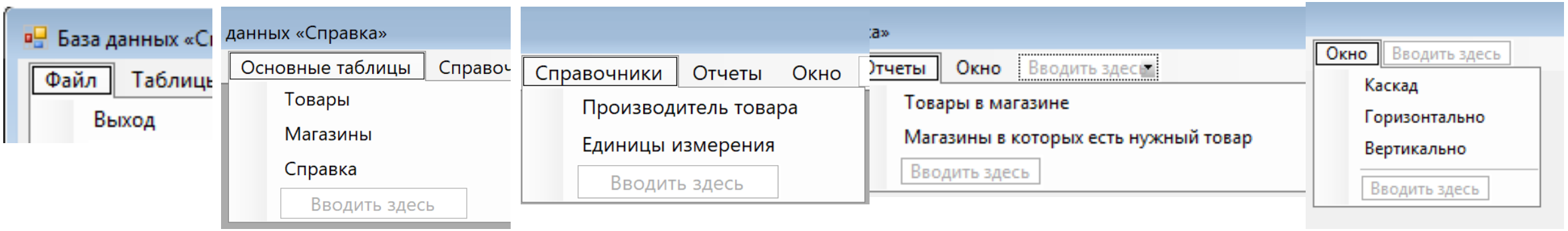
1. Создадим новое приложение.

2. Установим соответствующие свойства первой формы:

- **Name = FMDI**
- **IsMdiContainer = true**
- **Text = База данных «Справка»**
- **StartPosition = CenterScreen**

Это будет главная форма.

3. Разместим на форме компонент MenuStrip и спроектируем меню для главной формы следующим образом:



4. В меню ОКНО включим отображение имён всех открытых дочерних окон.

Для этого у компонента MenuStrip, представляющего контейнер главного меню, свойству MdiWindowListItem присвоим значение окноToolStripMenuItem – указание на пункт Окно.

5. Определим строку подключения к базе, для этого опишем соответствующее поле в классе основной формы:

```
private string ConnectionString = "Data Source=spravka.db;Version=3;";
```

6. Определим обработчики событий для пунктов меню

Файл – Выход:

```
private void ExitM_Click(object sender, EventArgs e)
{
    this.Close();
}
```

Окно – Каскад:

```
private void CascadM_Click(object sender, EventArgs e)
{
    LayoutMdi(MdiLayout.Cascade);
}
```

Окно – Горизонтально:

```
private void HorizontalM_Click(object sender, EventArgs e)
{
    LayoutMdi(MdiLayout.TileHorizontal);
}
```

Окно – Вертикально:

```
private void VerticalM_Click(object sender, EventArgs e)
{
    LayoutMdi(MdiLayout.TileVertical);
}
```

Таблицы – (Название таблицы):

Данный обработчик необходимо назначить для всех подпунктов меню.

```
private void EIzmerenM_Click(object sender, EventArgs e)
{
    string str = (sender as ToolStripMenuItem).Text;
```

```
    bool flag = true;
    foreach (var a in MdiChildren)
    {
        if (a.Text == str)
        {
            flag = false;
            break;
        }
    }
    if (flag)
    {
        Table t = new Table(str, ConnectionString);
        t.MdiParent = this;
        t.Show();
    }
}
```

7. Создадим дочернюю форму с именем Table

Разместим на ней компоненты

А) компонент toolStrip и создадим следующие кнопки:

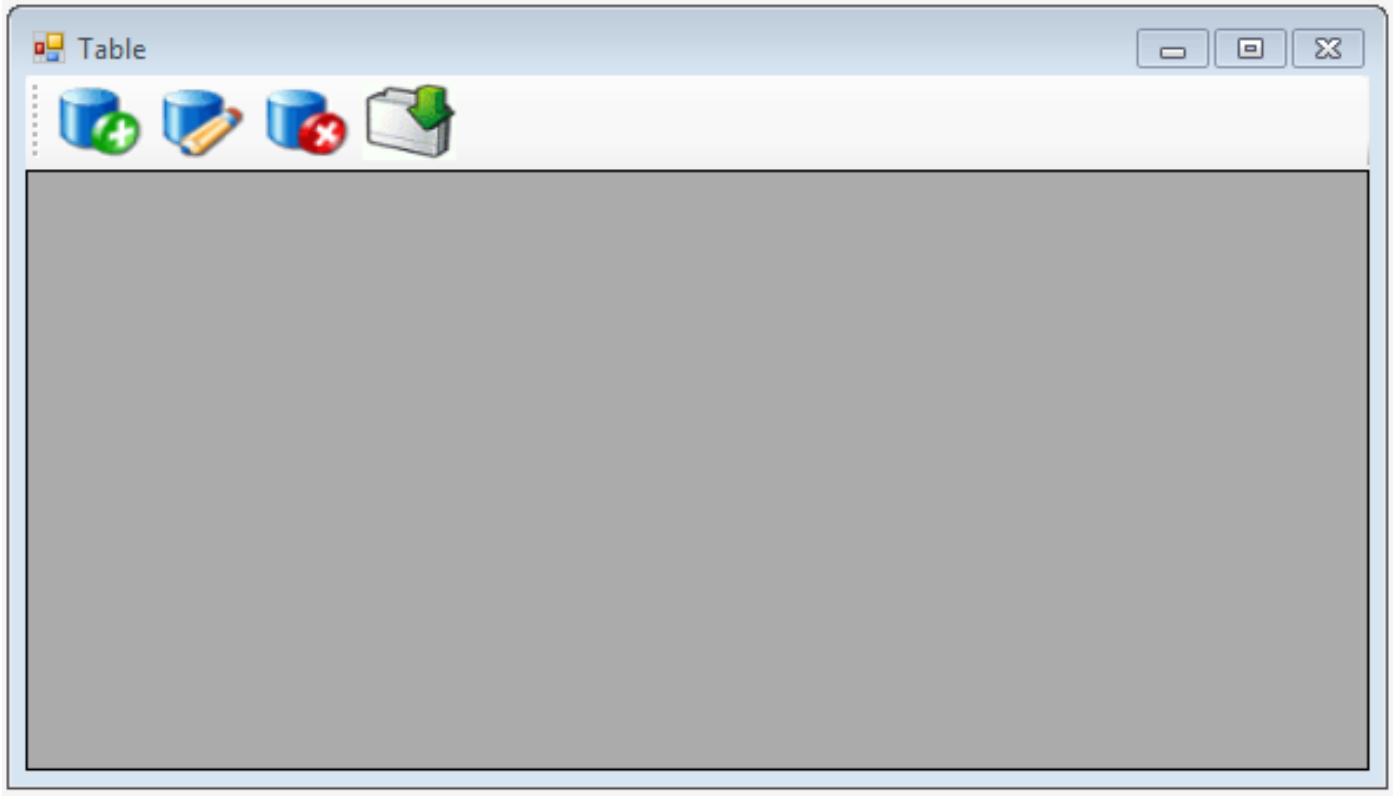
Name	Text
InsTable	Добавление данных
EditTable	Редактирование данных
DelTable	Удаление данных
CloseTable	Выход

Для каждой кнопки определим соответствующую картинку свойство: Image.

В результате получим следующую панель инструментов: 

Б) компонент DataGridView. Свойству Dock этого элемента установим значение Fill.

В результате получим следующую дочернюю форму:



Для отображения связанных таблиц спроектируем следующую форму:



8. Определим для работы с базой данных следующие поля:

```
private SQLiteConnection sc = new SQLiteConnection();
private SQLiteDataAdapter a;
private SQLiteCommand command = new SQLiteCommand();
private DataTable dt = new DataTable();

private string nameTable; //Название таблицы в базе данных
private string query; //Строка запроса

private int CurrentIndex = 0;  //Текущий индекс строки
```

9. Изменим конструктор дочерней формы следующим образом:

```
public Table(string nameF, string ConnectionString)
{
    InitializeComponent();
    this.Text= nameF;
    this.name = nameF;
    this.sc.ConnectionString = ConnectionString;
    switch (this.name)
    {
        case "Товары":
            {
                query = "SELECT Tovar.ID_T, Tovar.NameT AS [Название товара], Proizvod.ID_P,"+
                        " Proizvod.NameP AS [Название производителя]," +
                        " Proizvod.AdresP AS [Адрес производителя], "+
                        "EIzmeren.ID_E, EIzmeren.NameE AS [Единицы измерения]" +
                        " FROM EIzmeren INNER JOIN (Proizvod INNER JOIN Tovar ON Proizvod.ID_P"+
                        " = Tovar.FK_P) ON EIzmeren.ID_E = Tovar.FK_E; ";
                this.nameTable = "Tovar";
                break;
            }

            // Организуем выборку и для других таблиц

    }
```

```
    a = new SQLiteDataAdapter(query, sc);
    a.Fill(dt);
    dataGridView1.DataSource = dt;
    HideFields();
}
```

10. Определим следующие методы:

```
private void HideFields()
{
    switch (this.name)
    {
        case "Товары":
            {
                dataGridView1.Columns["ID_T"].Visible = false;
                dataGridView1.Columns["ID_E"].Visible = false;
                dataGridView1.Columns["ID_P"].Visible = false;
                break;
            }
        //Определим срытие полей и для других таблиц
    }
}
```

```
private void InsEdit(int param)
{
    int rowCount = dataGridView1.RowCount;
    if (rowCount != 0)
    {
        CurrentIndex = dataGridView1.CurrentRow.Index;
    }
    else
      if (param == 1)
    {
        MessageBox.Show("Нет данных для редактирования!"); return;
    }


        switch (name)
    {
        case "Товары":
            {
                InsEditTovar ed;
                if (param==0)  ed = new InsEditTovar("Добавление товара", this.sc.ConnectionString);
```

```
                else ed = new InsEditTovar("Изменение данных о товаре", this.sc.ConnectionString,
                                                dt.Rows[dataGridView1.CurrentRow.Index]);
                ed.Tag = param;
                ed.ShowDialog();
                break;
            }
            // Определим вызов окна Добавление/Редактирования и для других таблиц
    }

    dt.Clear();
    a = new SQLiteDataAdapter(query, sc);
    a.Fill(dt);
    dataGridView1.DataSource = dt;

    if (rowCount != 0)
    {
        dataGridView1.CurrentCell = rowCount == dataGridView1.RowCount ?
              dataGridView1.Rows[CurrentIndex].Cells[dataGridView1.ColumnCount - 1] :
              dataGridView1.Rows[rowCount].Cells[dataGridView1.ColumnCount - 1];
    }
}
```

11. Определим обработчики событий кнопок

Добавить:

```
private void toolStripInsTable_Click(object sender, EventArgs e)
{
    InsEdit(0);
}
```

Редактировать:

```
private void toolStripEditTable_Click(object sender, EventArgs e)
 {
     InsEdit(1);
 }
```

Выход:

```
private void toolStripCloseTable_Click(object sender, EventArgs e)
{
    Close();
}
```

Удалить:

```
private void toolStripDelTable_Click(object sender, EventArgs e)
{
    if (dataGridView1.RowCount == 0)
    {
        MessageBox.Show("Нет данных для удаления!"); return;
    }

    if (dataGridView1.RowCount != 0 && dataGridView1.CurrentRow.Index != 0)
    {
        CurrentIndex = dataGridView1.CurrentRow.Index - 1;
    }

    sc.Open();
    try
    {
            command = new SQLiteCommand("DELETE * FROM [" + nameTable + "] WHERE ["
                                        + dt.Columns[0].ColumnName +"] =" +
                                        dt.Rows[dataGridView1.CurrentRow.Index][0], sc);
            command.ExecuteNonQuery();
            dt.Clear();
```

```
            a = new SQLiteDataAdapter(query, sc);
            a.Fill(dt);
            dataGridView1.DataSource = dt;
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message);
        }
        finally
        {
            sc.Close();
        }

        if (dataGridView1.RowCount != 0 )
        {
            dataGridView1.CurrentCell =
                dataGridView1.Rows[CurrentIndex].Cells[dataGridView1.ColumnCount - 1];
        }

}
```

12. Создадим форму добавления и редактирования данных таблицы товар

А) Разместим на форме следующие компоненты:

- **3 компонента label;**

  label1.Text = Название товара:

  label2.Text = Ед. измерения:

  label3.Text = Производитель:

- **2 компонента comboBox для выбора единиц измерения и производителя;**

  comboBox1.DropDownStyle = DropDownList

  comboBox2.DropDownStyle = DropDownList

2 компонента button для выбора единиц измерения и производителя;

  button1.Text = ОК

  button2.Text = Отмены

- **1 компонент textBox для внесения названия товара.**

Б) Установим следующие свойства формы:

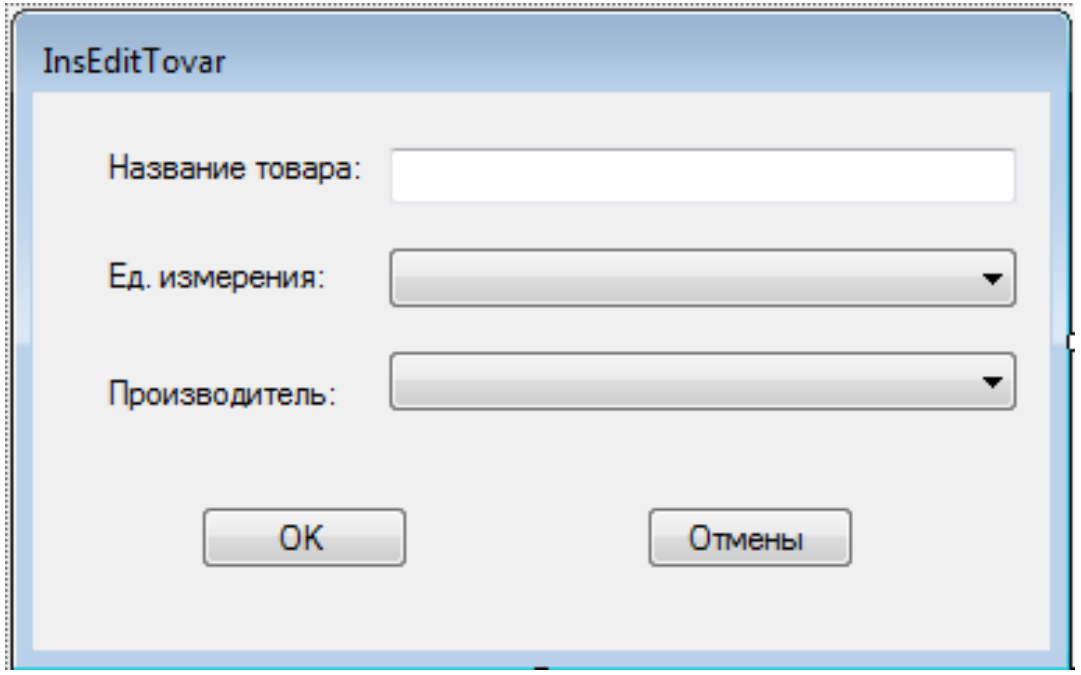
FormBorderStyle = FixedSingle

ControlBox = false

Name = InsEditTovar

StartPosition = CenterScreen

В результате получим следующую форму:



13. Определим для работы с базой данных следующие поля:

```
private SQLiteConnection sc;
private SQLiteCommand command;
private SQLiteDataAdapter a;
private DataSet tmp;
private DataRow dr;
```

14. Определим конструкторы формы при добавлении и при редактировании данных таблицы Товар:

А) При добавлении:

```
public InsEditTovar(string name, string ConnectionString)
{
    InitializeComponent();
    this.Text = name;
    sc = new SQLiteConnection();
    this.sc.ConnectionString = ConnectionString;
    tmp = new DataSet();
```

```
    tmp.Tables.Add(new DataTable("EIzmeren"));
    a = new SQLiteDataAdapter("SELECT ID_E, NameE FROM EIzmeren", sc);
    a.Fill(tmp.Tables["EIzmeren"]);

    comboBox1.DisplayMember = tmp.Tables["EIzmeren"].Columns["NameE"].ToString();
    comboBox1.ValueMember = tmp.Tables["EIzmeren"].Columns["ID_E"].ToString();
    comboBox1.DataSource = tmp.Tables["EIzmeren"];

    tmp.Tables.Add(new DataTable("Proizvod"));
    a = new SQLiteDataAdapter("SELECT ID_P, NameP FROM Proizvod", sc);
    a.Fill(tmp.Tables["Proizvod"]);

    comboBox2.DisplayMember = tmp.Tables["Proizvod"].Columns["NameP"].ToString();
    comboBox2.ValueMember = tmp.Tables["Proizvod"].Columns["ID_P"].ToString();
    comboBox2.DataSource = tmp.Tables["Proizvod"];

}
```

Б) При редактировании:

```
public InsEditTovar(string name, string ConnectionString, DataRow dr)
{
    InitializeComponent();
    this.Text = name;
    sc = new SQLiteConnection();
    this.sc.ConnectionString = ConnectionString;
    this.dr = dr;
    tmp = new DataSet();

    tmp.Tables.Add(new DataTable("EIzmeren"));
    a = new SQLiteDataAdapter("SELECT ID_E, NameE FROM EIzmeren", sc);
    a.Fill(tmp.Tables["EIzmeren"]);

    comboBox1.DisplayMember = tmp.Tables["EIzmeren"].Columns["NameE"].ToString();
    comboBox1.ValueMember = tmp.Tables["EIzmeren"].Columns["ID_E"].ToString();
    comboBox1.DataSource = tmp.Tables["EIzmeren"];

    tmp.Tables.Add(new DataTable("Proizvod"));
    a = new SQLiteDataAdapter("SELECT ID_P, NameP FROM Proizvod", sc);
    a.Fill(tmp.Tables["Proizvod"]);
```

```
    comboBox2.DisplayMember = tmp.Tables["Proizvod"].Columns["NameP"].ToString()
    comboBox2.ValueMember = tmp.Tables["Proizvod"].Columns["ID_P"].ToString();
    comboBox2.DataSource = tmp.Tables["Proizvod"];

    textBox1.Text = dr["Название товара"].ToString();
    comboBox1.SelectedValue = dr["ID_E"];
    comboBox2.SelectedValue = dr["ID_P"];

}
```

15. Напишем обработчики событий для кнопок:

А) Отмена:

```
private void button2_Click(object sender, EventArgs e)
{
    this.Close();
}
```

Б) ОК:

```
private void button1_Click(object sender, EventArgs e)
{
    if (comboBox1.Items.Count == 0 || comboBox2.Items.Count == 0
        || textBox1.Text.Count() == 0)
    {
        MessageBox.Show("Не все поля заполнены!");
    }
    else
    {
        sc.Open();

        if ((int)Tag == 0)
        {
            command = new SQLiteCommand("INSERT INTO Tovar (NameT, FK_E, FK_P) VALUES('"
              + textBox1.Text + "', '" + int.Parse(comboBox1.SelectedValue.ToString())
              + "', '" + int.Parse(comboBox2.SelectedValue.ToString()) + "')", sc);
        }
        else
        {
```

```
            command = new SQLiteCommand("UPDATE Tovar SET NameT = ? , FK_E = ? ,"
                +"FK_P = ?  WHERE ID_T = ?", sc);
            command.Parameters.AddWithValue("@NameT", textBox1.Text);
            command.Parameters.AddWithValue("@FK_E",
                int.Parse(comboBox1.SelectedValue.ToString()));
            command.Parameters.AddWithValue("@FK_P",
                int.Parse(comboBox2.SelectedValue.ToString()));
            command.Parameters.AddWithValue("@ID_T", dr[0]);
        }

        command.ExecuteNonQuery();
        sc.Close();
        Close();
    }
}
```

16. Для формирования отчетов создадим отдельную форму:

А) Разместим на форме следующие компоненты:

- **1 компонент label;**
- **1 компонент comboBox для выбора товара или магазина;**

  comboBox1.DropDownStyle = DropDownList
- **2 компонента button**

  button1.Text = Сформировать отчет

  button2.Text = Выход.

Б) Установим следующие свойства формы:

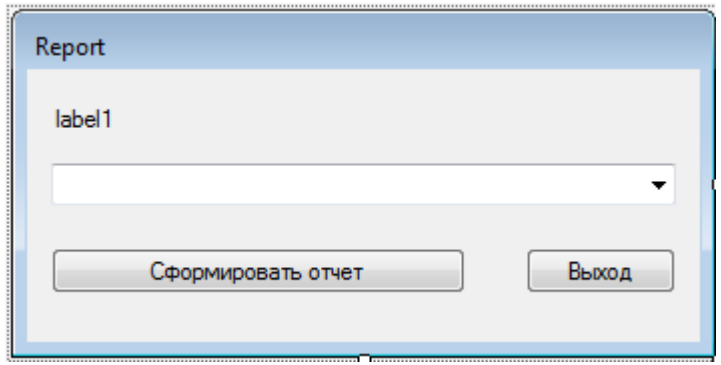
FormBorderStyle = FixedSingle

ControlBox = false

Name = Report

StartPosition = CenterScreen

В результате получим следующую форму:



17. Подключим офисные библиотеки.

А) В «Обозревателе решений» в разделе «Ссылки» щелкнем правой кнопкой мыши и выберем пункт «Добавить ссылку».

Б) На закладке «Com» выберем:

Microsoft Excel 16.0 Object Library

Microsoft Word 16.0 Object Library

В) В фале класса формы Report пропишем следующие директивы:

```
using Excel = Microsoft.Office.Interop.Excel;
using Word = Microsoft.Office.Interop.Word;
using System.Data.SQLite;
```

18. В главной форме создадим обработчики событий выбора необходимого отчета:

```
private void TovarinMagM_Click(object sender, EventArgs e)
{
    Report rep = new Report(1,ConnectionString);
    rep.Text = "Товары в магазине";
    rep.ShowDialog();
}

private void MaginTovarM_Click(object sender, EventArgs e)
{
    Report rep = new Report(2,ConnectionString);
    rep.Text = "Магазины в которых есть данный товар";
    rep.ShowDialog();
}
```

19. Определим для работы с базой данных следующие поля:

```
private SQLiteConnection sc = new SQLiteConnection();
private SQLiteDataAdapter a;
private DataSet tmp;
private DataTable dt = new DataTable();
private string query; //Строка запроса
```

20. Изменим конструктор формы следующим образом:

```
public Report(int rep, string ConnectionString)
{
    InitializeComponent();
    sc = new SQLiteConnection();
    this.sc.ConnectionString = ConnectionString;
    tmp = new DataSet();
    if (rep == 1)
```

```
{
    Tag = 1;
    label1.Text = "Выберите магазин:";
    tmp.Tables.Add(new DataTable("Magazin"));
    a = new SQLiteDataAdapter("SELECT Magazin.ID_M, Magazin.NameM +' ('+ Magazin.AdresM +')'"+
                              " As MagName FROM Magazin; ", sc);
    a.Fill(tmp.Tables["Magazin"]);

    comboBox1.DisplayMember = tmp.Tables["Magazin"].Columns["MagName"].ToString();
    comboBox1.ValueMember = tmp.Tables["Magazin"].Columns["ID_M"].ToString();
    comboBox1.DataSource = tmp.Tables["Magazin"];
}
else {
    Tag = 2;
    label1.Text = "Выберите товар:";
    tmp.Tables.Add(new DataTable("Tovar"));
    query = "SELECT Tovar.ID_T, Tovar.NameT +'('+ Proizvod.NameP +')' AS TovarName" +
            " FROM Proizvod INNER JOIN  Tovar  ON Proizvod.ID_P = Tovar.FK_P; ";
    a = new SQLiteDataAdapter(query, sc);
    a.Fill(tmp.Tables["Tovar"]);
```

```
            comboBox1.DisplayMember = tmp.Tables["Tovar"].Columns["TovarName"].ToString();
            comboBox1.ValueMember = tmp.Tables["Tovar"].Columns["ID_T"].ToString();
            comboBox1.DataSource = tmp.Tables["Tovar"];
        }
    }
```

21. Напишем обработчик для кнопки «Сформировать отчет»

```
private void button1_Click(object sender, EventArgs e)
{

    if ((int)Tag == 1)
    {
        query = "SELECT Tovar.NameT, Proizvod.NameP, Spravka.Kolvo, Spravka.Cena " +
                "FROM Proizvod INNER JOIN (Tovar INNER JOIN Spravka ON " +
                "Tovar.ID_T = Spravka.FK_T) ON Proizvod.ID_P = Tovar.FK_P " +
                "WHERE(Spravka.FK_M = " + int.Parse(comboBox1.SelectedValue.ToString()) + "); ";
        a = new SQLiteDataAdapter(query, sc);
        dt.Clear();
        a.Fill(dt);
```

```
if (dt.Rows.Count == 0)
{
    MessageBox.Show("Нет данных для формирования отчета!"); return;
}
else
{
    Excel.Application ObjExcel = new Excel.Application();
    Excel.Workbook ObjWorkBook;
    Excel.Worksheet ObjWorkSheet;
    //Создание книги
    ObjWorkBook = ObjExcel.Workbooks.Add(System.Reflection.Missing.Value);
    //Создание листа
    ObjWorkSheet = (Excel.Worksheet)ObjWorkBook.Sheets[1];
    ObjWorkSheet.Cells[1, 1] = "Магазин";
    ObjWorkSheet.Cells[1, 2] = comboBox1.Text;
    ObjWorkSheet.Cells[2, 1] = "Название товара";
    ObjWorkSheet.Cells[2, 2] = "Производитель";
    ObjWorkSheet.Cells[2, 3] = "Количество";
    ObjWorkSheet.Cells[2, 4] = "Цена";
```

```
        for (int _row = 0; _row < dt.Rows.Count; _row++)
        {
            {
                DataRow dr = dt.Rows[_row];
                ObjWorkSheet.Cells[_row + 3, 1] = dr["NameT"].ToString();
                ObjWorkSheet.Cells[_row + 3, 2] = dr["NameP"].ToString();
                ObjWorkSheet.Cells[_row + 3, 3] = dr["Kolvo"].ToString();
                ObjWorkSheet.Cells[_row + 3, 4] = dr["Cena"].ToString();
            }
        }
        ObjWorkBook.SaveAs("Отчет1.xlsx");
        ObjExcel.UserControl = true;
        ObjExcel.Quit();
        MessageBox.Show("Отчет сформирован!");
    }
}
else
{
    query = "SELECT Magazin.NameM, Magazin.AdresM, Spravka.Kolvo, Spravka.Cena "+
            "FROM Magazin INNER JOIN Spravka ON Magazin.ID_M = Spravka.FK_M "+
            "WHERE(Spravka.FK_T = " + int.Parse(comboBox1.SelectedValue.ToString()) + "); ";
```

```
a = new SQLiteDataAdapter(query, sc);
dt.Clear();
a.Fill(dt);
if (dt.Rows.Count == 0)
{
    MessageBox.Show("Нет данных для формирования отчета!"); return;
}
else
{
    Word.Application wrdApp = new Word.Application();
    Word.Document wrdDoc = wrdApp.Documents.Add();
    Word.Range range = wrdDoc.Range();
    range.InsertAfter("Товар: "+ comboBox1.Text);
    range.InsertParagraphAfter();
    range = wrdDoc.Range();
    Word.Table wrdTable = wrdDoc.Tables.Add(range, dt.Rows.Count + 1, 4);
    wrdTable.Borders.Enable = 1;

    wrdDoc.Tables[1].Cell(1, 1).Range.InsertAfter("Название магазина");
    wrdDoc.Tables[1].Cell(1, 2).Range.InsertAfter("Адрес магазина");
    wrdDoc.Tables[1].Cell(1, 3).Range.InsertAfter("Количество");
    wrdDoc.Tables[1].Cell(1, 4).Range.InsertAfter("Цена");
```

```
            for (int _row = 0; _row < dt.Rows.Count; _row++)
            {
                DataRow dr = dt.Rows[_row];
                wrdDoc.Tables[1].Cell(_row + 2, 1).Range.InsertAfter(dr["NameM"].ToString());
                wrdDoc.Tables[1].Cell(_row + 2, 2).Range.InsertAfter(dr["AdresM"].ToString());
                wrdDoc.Tables[1].Cell(_row + 2, 3).Range.InsertAfter(dr["Kolvo"].ToString());
                wrdDoc.Tables[1].Cell(_row + 2, 4).Range.InsertAfter(dr["Cena"].ToString());
            }
            wrdDoc.Saved = true;
            wrdDoc.SaveAs("Отчет2.docx");
            wrdDoc.Close();
            wrdApp.Quit();
            wrdDoc = null;
            wrdApp = null;
            MessageBox.Show("Отчет сформирован!");
        }
    }
}
```

22. Напишем обработчик для кнопки «Выход»

```
private void button2_Click(object sender, EventArgs e)
{
    this.Close();
}
```

